Computer Architecture
Instructions: Language of the Computer (II)

Tatpong Katanyukul

CoE, KKU

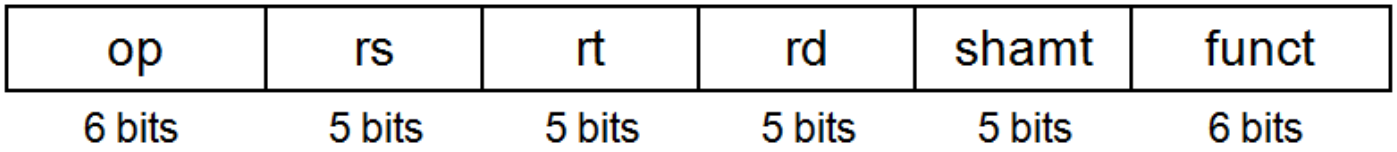
2 พฤษภาคม พ.ศ. 2560

Logical Operations

Operation	C	Java	MIPS
Shift left	<<	<<	sll
Shift right	>>	>>	srl
Bitwise AND	&	&	and, andi
Bitwise OR	|	|	or, ori
Bitwise NOT	~	~	nor

Category	Instruction	Example	Meaning	Comments
Logical	and	and $s1,$s2,$s3	$s1 = $s2 & $s3	Three reg. operands; bit-by-bit AND
	or	or $s1,$s2,$s3	$s1 = $s2 | $s3	Three reg. operands; bit-by-bit OR
	nor	nor $s1,$s2,$s3	$s1 = ~ ($s2 | $s3)	Three reg. operands; bit-by-bit NOR
	and immediate	andi $s1,$s2,20	$s1 = $s2 & 20	Bit-by-bit AND reg with constant
	or immediate	ori $s1,$s2,20	$s1 = $s2 | 20	Bit-by-bit OR reg with constant
	shift left logical	sll $s1,$s2,10	$s1 = $s2 << 10	Shift left by constant
	shift right logical	srl $s1,$s2,10	$s1 = $s2 >> 10	Shift right by constant

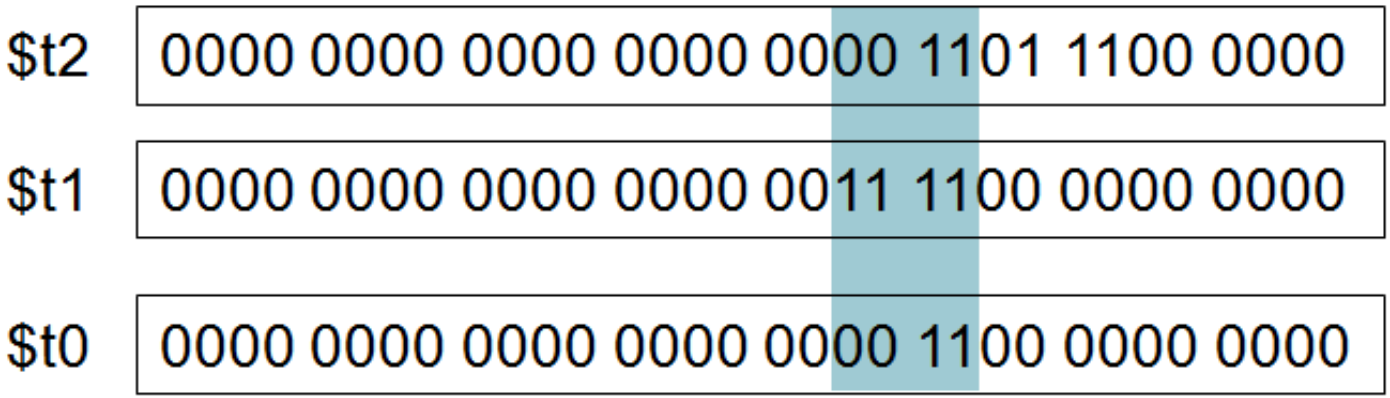
Shift Operations



- shamt: how many positions to shift
- Shift left logical
  - Shift left and fill with 0 bits
  - sll by i bits $\equiv$ multiplies by 2^i
- Shift right logical
  - Shift right and fill bits with msb's value
  - srl by i bits

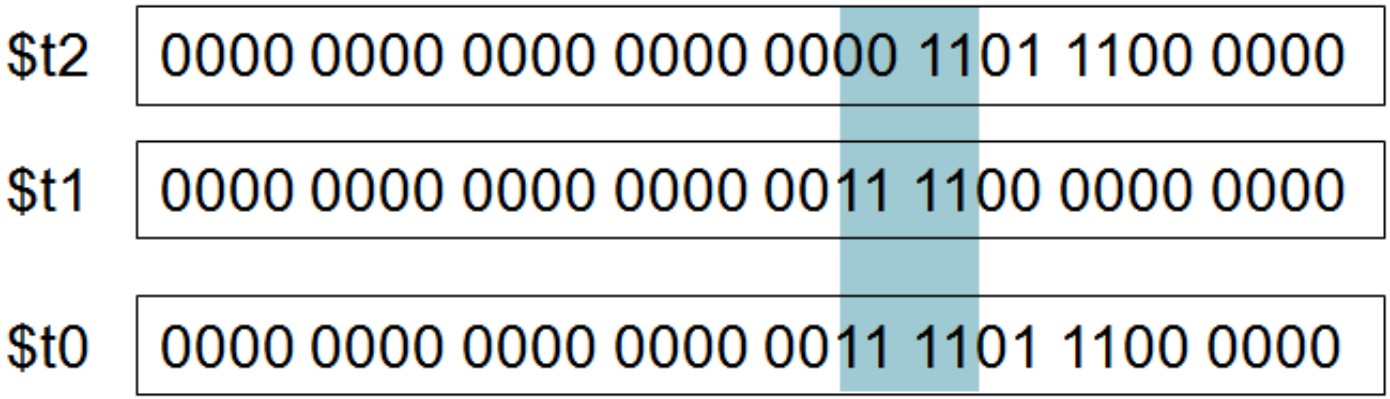
AND Operations

- Useful to mask bits in a word
- and \$t0, \$t1, \$t2



OR Operations

- Useful to include bits in a word
  - Set some bits to 1, leave others unchanged
- or \$t0, \$t1, \$t2



NOT Operations

- Useful to invert bits in a word
  - Change 0 to 1, and 1 to 0
- MIPS has NOR 3-operand instruction

  a NOR $b \equiv$ NOT (a OR b)
- nor \$t0, \$t1, \$zero

\$t1	0000 0000 0000 0000 0011 1100 0000 0000
\$t0	1111 1111 1111 1111 1100 0011 1111 1111

Instructions for Making Decisions

Category	Instruction	Example	Meaning	Comments
Conditional branch	branch on equal	`beq $s1,$s2,25`	if ($s1 == $s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	`bne $s1,$s2,25`	if ($s1!= $s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	`slt $s1,$s2,$s3`	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; for `beq`, `bne`
	set on less than unsigned	`sltu $s1,$s2,$s3`	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than unsigned
	set less than immediate	`slti $s1,$s2,20`	if ($s2 < 20) $s1 = 1; else $s1 = 0	Compare less than constant
	set less than immediate unsigned	`sltiu $s1,$s2,20`	if ($s2 < 20) $s1 = 1; else $s1 = 0	Compare less than constant unsigned
Unconditional jump	jump	`j 2500`	go to 10000	Jump to target address
	jump register	`jr $ra`	go to $ra	For switch, procedure return
	jump and link	`jal 2500`	$ra = PC + 4; go to 10000	For procedure call

Conditional Operations

- Branch to a labeled instruction if a condition is true
  - Otherwise, continue sequentially
- beq rs, rt, L1
  - if (rs == rt) branch to instruction labeled L1;
- bne rs, rt, L1
  - if (rs != rt) branch to instruction labeled L1;
- j L1
  - unconditional jump to instruction labeled L1

Compiling If Statements

- C code:

```
if (i==j) f = g+h;
else f = g-h;
```

- f, g, h, i, and j
  $\Rightarrow$ \$s0, \$s1, \$s2, \$s3, \$s4

Compiled MIPS code:

```
      bne $s3, $s4, Else
      add $s0, $s1, $s2
      j Exit
Else: sub $s0, $s1, $s2
Exit: ...
```

Assembler calculates addresses: Else and Exit

Compiling Loop Statements

- C code:

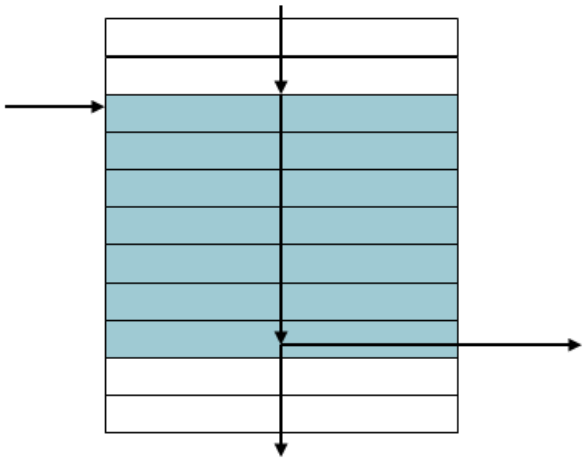
```
while (save[i] == k) i += 1;
```

  - i in $s3, k in $s5, base address of save in $s6

- Compiled MIPS code:

```
Loop: sll  $t1, $s3, 2
      add  $t1, $t1, $s6
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: ...
```

Basic Blocks



จากตัวอย่าง Loop Statements

```
Loop:   sll   $t1, $s3, 2
        add   $t1, $t1, $s6
        lw    $t0, 0($t1)
        bne   $t0, $s5, Exit
```

- A basic block is a sequence of instructions with
  - No embedded branches (except at end)
  - No branch targets (except at beginning)
- A compiler identifies basic blocks for optimization

More Conditional Operations

- Set result to 1 if a condition is true
  - Otherwise, set to 0
- slt rd, rs, rt
  - if (rs < rt) then rd = 1; else rd = 0;
- slti rt, rs, constant
  - if (rs < constant) rt = 1; else rt = 0;
- Use in combination with beq, bne

```
slt $t0, $s1, $s2    # if ($s1 < $s2)
bne $t0, $zero, L1   #     branch to L1
```

Branch Instruction Design

- Why not blt, bge, etc?
- Hardware for $<$, $\geq$, ... slower than $=$, $\neq$
  - Combining with branch involves more work per instruction.
    More work per instruction $\Rightarrow$ requiring a slower clock
    $\Rightarrow$ All instructions are penalized
- beq and bne are the common case
- This is a good design compromise

Performing Other Conditional Operations

Suppose \$s0 and \$s1 are assigned to x and y, respectively,

$x < y$	slt \$t0, \$s0, \$s1
$x \geq y$	

Performing Other Conditional Operations

Suppose \$s0 and \$s1 are assigned to x and y, respectively,

$x < y$	slt \$t0, \$s0, \$s1
$x \geq y \equiv$ "*not*" $(x < y)$	slt \$t0, \$s0, \$s1 ทำ "not on lsb" เช่น slti \$t0, \$t0, 1 หรือ addi \$t1, \$zero, 1 ตามด้วย xor \$t0, \$t0, \$t1 แต่ nor \$t0, \$t0, \$zero $\equiv$ not on every bit (หรือ เลือกใช้ beq / bne ให้เหมาะสม)
$y < x$	

Performing Other Conditional Operations

Suppose \$s0 and \$s1 are assigned to x and y, respectively,

$x < y$	slt \$t0, \$s0, \$s1
$x \geq y \equiv$ "*not*" $(x < y)$	slt \$t0, \$s0, \$s1 ทำ "not on lsb" เช่น slti \$t0, \$t0, 1 หรือ addi \$t1, \$zero, 1 ตามด้วย xor \$t0, \$t0, \$t1 แต่ nor \$t0, \$t0, \$zero $\equiv$ not on every bit (หรือ เลือกใช้ beq / bne ให้เหมาะสม)
$y < x$	slt \$t0, \$s1, \$s0
$y \geq x \equiv not(y < x)$	slt \$t0, \$s1, \$s0 slti \$t0, \$t0, 1 (หรือ เลือกใช้ beq / bne ให้เหมาะสม)

Performing Other Conditional Operations

Suppose \$s0 and \$s1 are assigned to x and y, respectively,

$x \neq y \equiv (x < y) or (y < x)$	slt \$t1, \$s0, \$s1 slt \$t2, \$s1, \$s0 or \$t0, \$t1, \$t2 (หรือ เลือกใช้ beq / bne ให้เหมาะสม)
$x = y \equiv$ "not" $x \neq y$	slt \$t1, \$s0, \$s1 slt \$t2, \$s1, \$s0 or \$t0, \$t1, \$t2 slti \$t0, \$t0, 1 (หรือ เลือกใช้ beq / bne ให้เหมาะสม)

Compiling Logical Assignment

C Code:

```
b = 24;
c = 35;
d = 16;
a = (b == c);
c = (d != b);
```

MIPS Assembly:

a, b, c, d $\Rightarrow$ $s1, $s2, $s3, $s4

Compiling Logical Assignment

C Code:

```
b = 24;
c = 35;
d = 16;
a = (b == c);
c = (d != b);
```

MIPS Assembly:

a, b, c, d ⇒ $s1, $s2, $s3, $s4

```
ori $s1, $zero, 24
ori $s2, $zero, 35
ori $s3, $zero, 16
xor $t0,$s2,$s3   ##  b xor c
sltiu $s1,$t0,1   ##  a = (b == c)
xor     $t0,$s4,$s2   ##  d xor b
sltu $s1,$0,$t0   ##  a = (d != b)
```

Signed vs. Unsigned

- Signed comparison: slt, slti
- Unsigned comparison: sltu, sltui
- Example
  - $s0 = 1111 1111 1111 1111 1111 1111 1111 1111
  - $s1 = 0000 0000 0000 0000 0000 0000 0000 0001
  - slt $t0, $s0, $s1 # signed
    - $-1 < +1 \Rightarrow$ $t0 = 1
  - sltu $t0, $s0, $s1 # unsigned
    - $+4,294,967,295 > +1 \Rightarrow$ $t0 = 0

Bounds Check Shortcut

- Index of an array goes from 0 to $N - 1$, where N is size of the array.
- Checking index out of bound in one shot.

Bounds Check Shortcut

- Index of an array goes from 0 to $N - 1$, where N is size of the array.
- Checking index out of bound in one shot.
  - E.g., int A[10];
  - check if i would be good with A[i].
  - Let $s0 be assigned to i.
  - sltiu $t0, $s0, 10
    - if i = 11 $\Rightarrow 11 > 10 \Rightarrow$ $t0 = 0
    - if i = 10 $\Rightarrow 10 = 10 \Rightarrow$ $t0 = 0
    - if i = 9 $\Rightarrow 9 < 10 \Rightarrow$ $t0 = 1
    - ...
    - if i = 0 $\Rightarrow 0 < 10 \Rightarrow$ $t0 = 1
    - if i = -1 $\Rightarrow 4,294,967,295 > 10 \Rightarrow$ $t0 = 0

Index Out of Bound: C v.s. Java

```
1  // tryIndexArray.c
2  #include <stdio.h>
3
4  main(){
5     int A[10];
6     int i;
7
8     for(i=0;i<10;i++) printf("A[%d] = %d\n", i, A[i]);
9     printf("\n\n");
10
11    A[0] = 138;
12    A[10] = 165;
13    for(i=0;i<=10;i++) printf("A[%d] = %d\n", i, A[i]);
14 }
```

- C would let this slip out.

Index Out of Bound: C v.s. Java

```
1  // tryIndexArray.java
2  public class tryIndexArray {
3  public static void main(String args[])
4  {
5     int A[] = new int[10];
6     int i;
7
8     for(i=0;i<10;i++)
9        System.out.println("A[" + i + "] =↩
               " + A[i]);
10    System.out.println("\n");
11    A[0] = 138;
12    A[10] = 165;
13    for(i=0;i<=10;i++)
14       System.out.println("A[↩
               " + i + "] = " + A↩
               [i]);
15 }
16 }
```

- Java does not let you get away with it.

For Loop Example

C code:

```
for(i = 0; i < N; i++)
  A[i] = i
```

MIPS assembly:

$s0, $s1, $s2 ⇒ i, base address of array A, and N.

```
        add   $s0, $zero, $zero     ## i = 0
FL1:    sltu  $t0, $s0, $s2         ## check if i < N
        beq   $t0, $zero, IndexOutOfBound
        sll   $t0, $s0, 2           ## 4 bytes / word
        add   $t0, $t0, $s1         ## A[i]'s address
        sw    $s0, 0($t0)           ## A[i] = i
        addi  $s0, $s0, 1           ## i++
        j     FL1
IndexOutOfBound: ...
```

Supporting Procedures

- Place parameters in registers:
  procedure arguments $\rightarrow$ registers
- Transfer control to procedure:
  procedure address $\rightarrow$ PC
- Acquire storage for procedure
- Perform procedure's operations
- Place result in register for caller:
  results $\rightarrow$ registers
- Return to place of call:
  address of the caller's following instruction $\rightarrow$ PC

Supporting Procedures: Register Usage

```
int leaf_example (int g, h, i, j){
  int f;
  f = (g + h) - (i + j);
  return f;
}
```

- $a0 – $a3: arguments (reg's 4 – 7)
- $v0, $v1: result values (reg's 2 and 3)
- $t0 — $t9: temporaries
  Can be overwritten by callee
- $s0 -– $s7: saved
  Must be saved/restored by callee
- $sp: stack pointer (reg 29)
- $ra: return address (reg 31)

Procedure Call Instructions

- Procedure call: jump and link jal ProcedureLabel
  - Address of following instruction put in $ra
  - Jumps to target address
- Procedure return: jump register jr $ra
  - Copies $ra to program counter (PC)

Leaf Procedure Example

```
int leaf_example (int g, h, i, j){
  int f;
  f = (g + h) - (i + j);
  return f;
}
```

- Arguments g, h, i, j in $a0, $a1, $a2, $a3
- f in $s0 (hence, need to save $s0 on stack)
- Result in $v0

Leaf Procedure Example

C code:

```
int leafExample (int g, h, i, j){
  int f;
  f = (g + h) - (i + j);
  return f;
}
```

MIPS Code:

```
leafExample:
addi $sp, $sp, -4
sw   $s0, 0($sp)    # save $s0

add  $t0, $a0, $a1 # g + h
add  $t1, $a2, $a3 # i + j
sub  $s0, $t0, $t1 # f=(g+h)-(i+j)

add  $v0, $s0, $0  # result

lw   $s0, 0($sp)    # restore $s0
addi $sp, $sp, 4

jr   $ra            # back to caller
```

Non-Leaf Procedures

- Procedures that call other procedures
- For nested call, caller needs to save on the stack:
  - Its return address
  - Any arguments and temporaries needed after the call

  Restore from the stack after the call

Non-Leaf Procedure Example

C Code:

```c
int fact (int n)
{
  if (n < 1) return 1;
  else return n * fact(n - 1);
}
```

- Argument n in $a0
- Result in $v0

MIPS Code:

```
fact:
    addi $sp, $sp, -8
    sw   $ra, 4($sp)      # push $ra
    sw   $a0, 0($sp)      # push $a0
    slti $t0, $a0, 1      # (n < 1)?
    beq  $t0, $zero, L1
    addi $v0, $zero, 1    # fact(0)=1
    addi $sp, $sp, 8
    jr   $ra
L1: addi $a0, $a0, -1     # n - 1
    jal  fact             # fact(n-1)
    lw   $a0, 0($sp)      # pop $a0
    lw   $ra, 4($sp)      # pop $ra
    addi $sp, $sp, 8
    mul  $v0, $a0, $v0
    jr   $ra
```

Stack

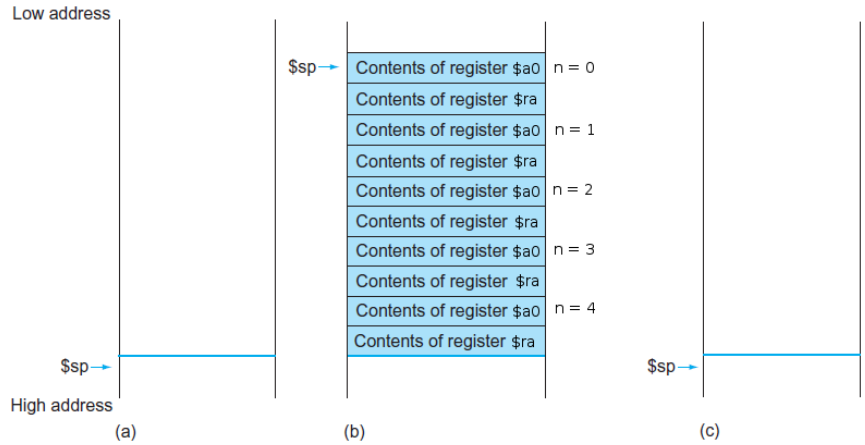
Data | Text

Data

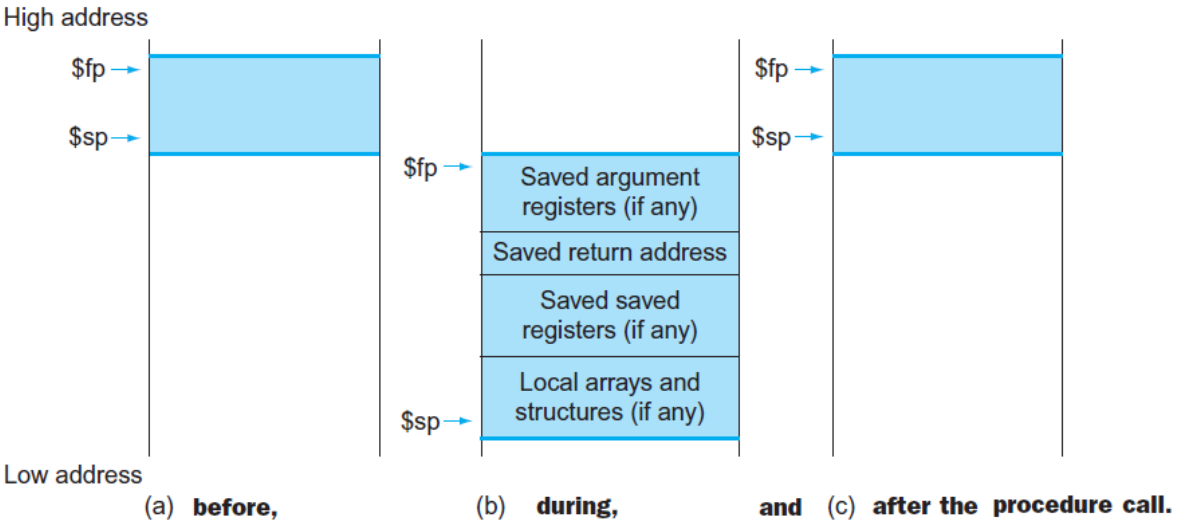
```
User data segment [10000000]..[10040000]
[10000000]..[1003ffff]  00000000


User Stack [7ffff8bc]..[80000000]
[7ffff8bc]    00000000
[7ffff8c0]    0040004c  00000001  0040004c  00000002
[7ffff8d0]    0040004c  00000003  0040004c  00000004
[7ffff8e0]    00400068  00000001  7ffff97b  00000000
[7ffff8f0]    7ffffe1  7fffffbd  7fffffaa  7fffff79
[7ffff900]    7fffff66  7fffff42  7fffff2e  7fffff21
[7ffff910]    7fffff0e  7ffffee7  7ffffed3  7ffffebc
[7ffff920]    7ffffeae  7ffffbde  7ffffba0  7ffffb85
[7ffff930]    7ffffb41  7ffffb2f  7ffffb17  7ffffafc
[7ffff940]    7ffffade  7ffffa9d  7ffffa86  7ffffa72
[7ffff950]    7ffffa63  7ffffa4d  7ffffa29  7ffffa06
[7ffff960]    7ffff9f5  7ffff9e9  7ffff9d1  7ffff9bf
[7ffff970]    00000000  00000000  44000000  6c432f3a
[7ffff980]    65737361  31302f73  414f435f  3130322f
[7ffff990]    2f325434  64696c53  432f7365  6e493230
[7ffff9a0]    75727473  6f697463  2f42736e  646e6148
[7ffff9b0]    2f6e4f73  74636166  612e3130  77006d73
[7ffff9c0]    69646e69  3a433d72  6e69775c  73776f64
```

```
R6  [a2] = 7ffff8f0
R7  [a3] = 0
R8  [t0] = 0
R9  [t1] = 0
R10 [t2] = 0
R11 [t3] = 0
R12 [t4] = 0
R13 [t5] = 0
R14 [t6] = 0
R15 [t7] = 0
R16 [s0] = 0
R17 [s1] = 0
R18 [s2] = 0
R19 [s3] = 0
R20 [s4] = 0
R21 [s5] = 0
R22 [s6] = 0
R23 [s7] = 0
R24 [t8] = 0
R25 [t9] = 0
R26 [k0] = 0
R27 [k1] = 0
R28 [gp] = 10008000
R29 [sp] = 7ffff8bc
R30 [s8] = 0
R31 [ra] = 40004c
```



Local Data on the Stack



- stack is also used to store local variables

  E.g., local arrays or structures
- stack may change within the procedure.
  - $sp points to the end of stack (last word of data)
  - When stack grows, $sp is changed.
- For convenience, $fp points to the beginning of stack

  $fp is held constant throughout the procedure.

Remark on Frame Pointer

- If the procedure has more than 4 arguments,
  the convention is to place the extra on the stack, just above the frame pointer.
- Frame pointer is convenient, but not necessary.
  - GNU MIPS C compiler uses a frame pointer
  - But, C compiler from MIPS does not.
    Register 30 is treated as $s8.

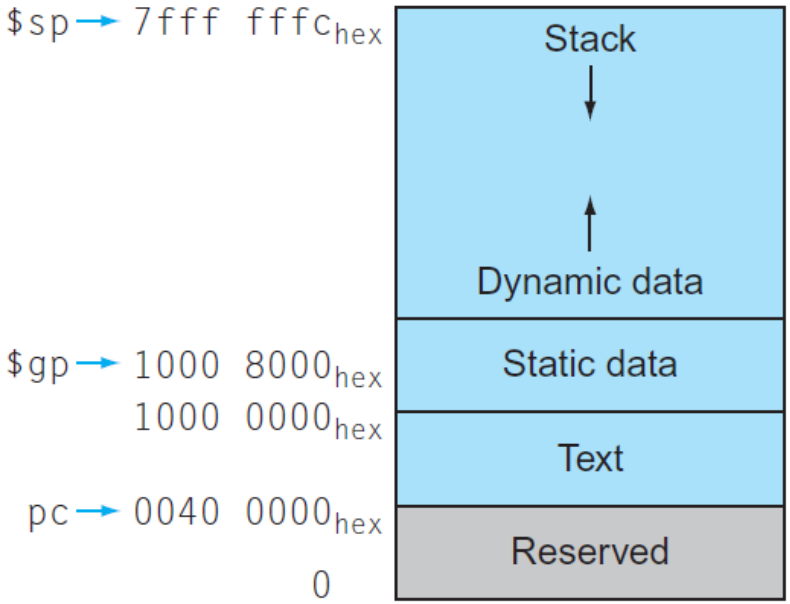
Remark on Recursive Procedure

- Some recursive procedure can be implemented with iteration.
- Iteration improves performance: no overhead of recursive procedure calls.

C Code:

```
int ifact (int n)
{
  int i;
  int f;
  f = 1;
  for(i=1;i<=n;i++){
    f = f*i;
  }
return(f);
}
```

Conventional Memory Layout



- Text segment: for MIPS Machine Code
- Static data: for constants and static variables
- Dynamic data: for dynamic data a.k.a. “heap”

Examples of C Static Data

```
const float pi=3.1416; // constant

main(){
  int x;          // static variable
  int A[4];       // array

  struct dog {    // structure
    int age;
    char sex;
    char breed[50];
  } Hugo;

  Hugo.age = 2;
  shiro.sex = 'm';
  strcpy(shiro.breed, "X-Bred");
   :      :      :      :      :
```

- Constant pi, static variable x, array A, structure Hugo $\Rightarrow$ static space

Examples of C Dynamic Data

```
void main(){
 char boxer[50];
 char *desc;
 strcpy(boxer, "Buakao");
 desc = malloc(100*sizeof(char));
 if( desc == NULL ) {
  printf(stderr, "Not enough memory\n");
  return;
 } else {
  strcpy(desc, "X Fight Champion");
 }

 printf("boxer %s\n", boxer);
 printf("%s\n\n", desc);

 free(desc);
}
```

- C allocates space with malloc and frees with free. reallocation: realloc
- Java uses only new to allocate space and relies on garbage collection for cleaning up.

Character Data

- Most computers today use 8-bit bytes to represent characters, with the American Standard Code for Information Interchange (ASCII)

ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter
32	space	48	0	64	@	80	P	96	`	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	|
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

ASCII Characters

- Upper- and lowercase letters differ by 32.
  shortcut in checking or changing upper- and lowercase.
- ASCII value of 0 represents null: C uses null to mark the end of a string

ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter
32	space	48	0	64	@	80	P	96	`	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	|
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

Load/Store Byte Instructions

- Text is widely used.

  MIPS provides instructions to move bytes:

  - lb $t0, 0($s1)
  - sb $t0, 0($s1)

MIPS Code (with QtSPIM directive): move text

```
1     .data 0x10000000
2     .asciiz "Aloha, world."
3     .text
4     .globl main
5  main: li $s0, 0x10000000 ## src addr.
6        li $s1, 0x100000a0 ## des addr.
7        li $s2, 0          ## i
8  L1: add $t0, $s0, $s2
9     lb  $t1,0($t0)
10    add $t0, $s1, $s2
11    sb  $t1,0($t0)
12    addi $s2, $s2, 1   ## i++
13    bne $t1, $0, L1
14    nop
```

Strings

Characters make up a string: Representing a string in C and Java

- C: last position of string is indicated by a character to mark ending, null (0x00).
  E.g., “Cal”: 4 bytes $\Rightarrow$ 67, 97, 108, 0.
- Java: first position is reserved for the length of a string.

32-bit Constants

- Most constants are small
  - 16-bit immediate is sufficient
- For the occasional 32-bit constant

  lui rt, constant

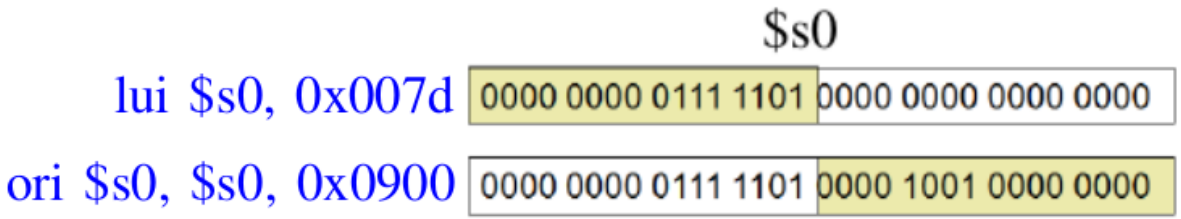
  I-Format:

001111	00000	rt	imm

  - Copies 16-bit constant to left 16 bits of rt
  - Clears right 16 bits of rt to 0
  - Example

    li $s0, 0x007d0900 (pseudo instruction)

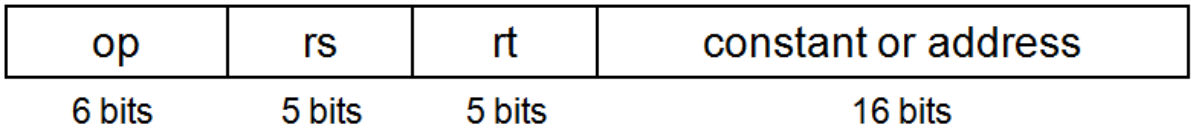
    ≡ lui $s0, 0x007d ; ori $s0, $s0, 0x0900



Branch Addressing

- Branch instructions specify
  - Opcode, two registers, target address
- Most branch targets are near branch
  - Forward or backward

I-Format:



PC-relative addressing

- Target address = PC + offset $\times 4$
- PC already incremented by 4 by this time

Branch Addressing: Example

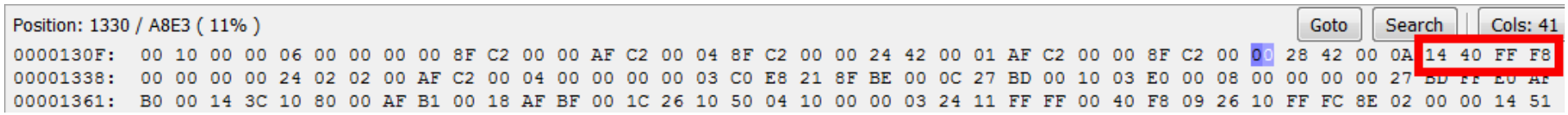
Branch to 0x00400038 from 0x00400054

$$\begin{aligned} \text{Target} &= (\text{PC}+4) + \text{offset} \times 4 \\ \therefore \text{offset} &= (\text{Target} - (\text{PC}+4))/4 \\ &= (0\text{x}00400038 - 0\text{x}00400058)/4 = -8 = 0\text{xfff8} \end{aligned}$$

QtSPIM 9.1.12 does not get branch offset exactly right.

```
[00400038] 8fc20000  lw $2, 0($30)              ; 13: lw $2,0($fp) ## L3 starts here
[0040003c] afc20004  sw $2, 4($30)              ; 14: sw $2,4($fp)
[00400040] 8fc20000  lw $2, 0($30)              ; 15: lw $2,0($fp)
[00400044] 24420001  addiu $2, $2, 1            ; 16: addiu $2,$2,1
[00400048] afc20000  sw $2, 0($30)              ; 17: sw $2,0($fp)
[0040004c] 8fc20000  lw $2, 0($30)              ; 19: lw $2,0($fp)
[00400050] 2842000a  slti $2, $2, 10            ; 20: slt $2,$2,10
[00400054] 1440fff9  bne $2, $0, -28 [.L3-0x00400054]
[00400058] 00000000  nop                        ; 22: nop
```

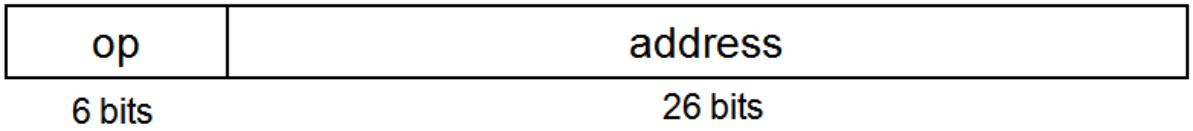
C.f. object code by Sourcery CodeBench Lite: MIPS ELF (2012.09-98)



Jump Addressing

- Jump (j and jal) targets could be anywhere in text segment

  J Format:



- Ideal: PC = JumpAddr
- Reality: (Pseudo)Direct jump addressing

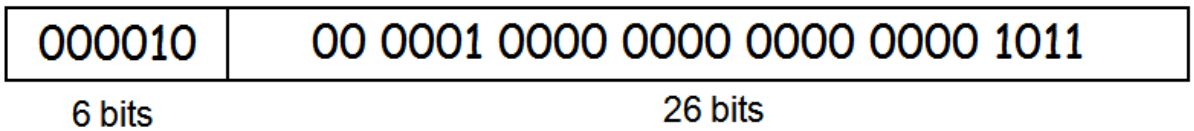
$$\text{Target address} = \text{PC}_{31\ldots28} : (\text{address} \times 4)$$

Jump Addressing: Example

```
[00400024] 34130000  ori $19, $0, 0                    ; 6: li $s3, 0 # i = 0
[00400028] 3c161000  lui $22, 4096                     ; 7: li $s6, 0x10000000 # Array A
[0040002c] 00134880  sll $9, $19, 2                    ; 9: sll $t1, $s3, 2 # L1
[00400030] 01364820  add $9, $9, $22                   ; 10: add $t1, $t1, $s6
[00400034] 8d280000  lw $8, 0($9)                      ; 11: lw $t0, 0($t1)
[00400038] 15150003  bne $8, $21, 12 [Exit-0x00400038]
[0040003c] 22730001  addi $19, $19, 1                  ; 13: addi $s3, $s3, 1
[00400040] 0810000b  j 0x0040002c [L1]                 ; 14: j L1
[00400044] 00000000  nop                               ; 16: nop # Exit
```

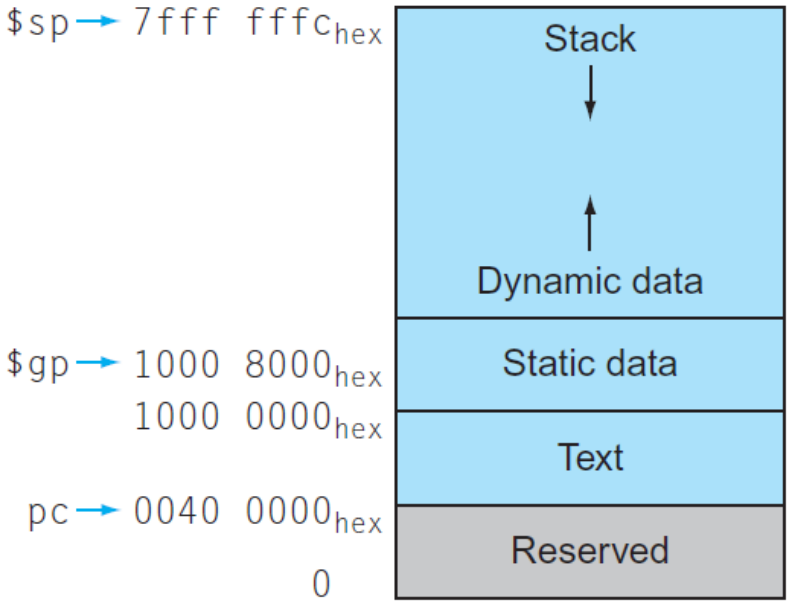
- at address 0x00400040, jump to address 0x0040002c

$$\begin{aligned} \text{Target address} &= \text{PC}_{31\ldots28} : (\text{address} \times 4) \\ \therefore \text{address field} &= \text{Target address}_{27\ldots0}/4 \\ &= 0\text{x}040002\text{c}/4 = 0\text{x}010000\text{b} \end{aligned}$$



Jump Addressing

$$\text{Target address} = \text{PC}_{31\ldots28} : (\text{address} \times 4)$$



Notice Memory Layout Convention:
If we follow the convention, program will be within address range
0x00400000 – 0x0FFFFFFF
($\text{PC}_{31\ldots28}$ is always $0x0 = 0000_2$).

Jump Addressing

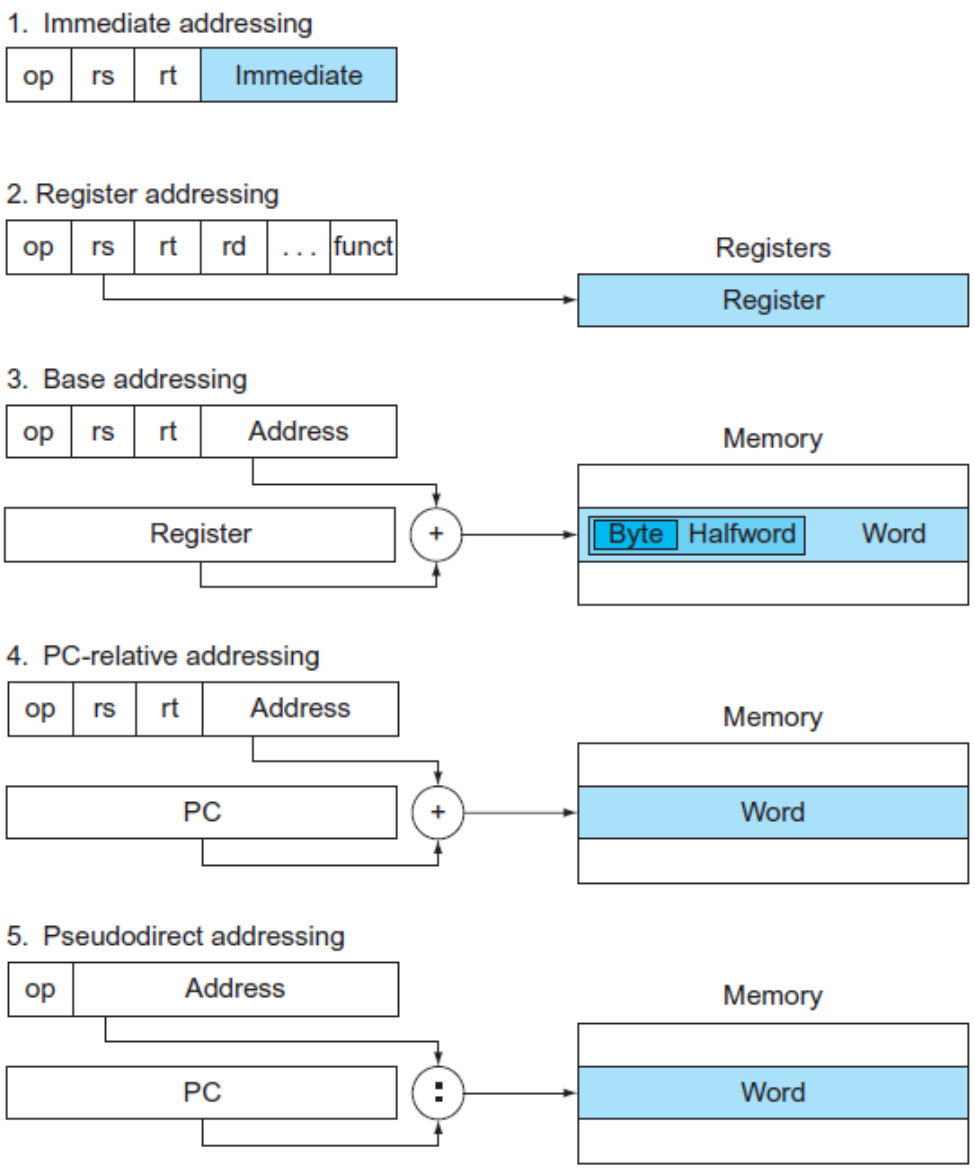
- If a program can fit in 256MB (2^{28} jumping range), pseudo direct addressing will work perfectly.
- The loader and linker must be careful to avoid placing a program across an address boundary of 256MB ($\approx$ 67 million instructions).
- any jump requires to go over the boundary must be replaced by: loading the target address to a register, li rt, Addr
  and jump register instruction, jr rt.

Branching Far Away

If branch target is too far to encode with 16-bit offset, assembler rewrites the code:

beq $s0, $s1, L1		bne $s0, $s1, L2
...	$\Rightarrow$	j L1
...		L2: ...

Addressing Mode Summary



- Immediate mode, e.g., addi
- register mode, e.g., add
- base mode, e.g., lw
- PC-relative mode, e.g., bne
- Pseudodirect mode, e.g., j

Closing note

- อ่าน P&H5e, 2.6-2.10.

